

UNIT-III

Dictionaries, Tuples, and Sets

Experiment 13: Tuple Creation and Concatenation

Aim: To write a Python program that creates tuples with student information (name, age, address, college) for at least two students, concatenates these tuples, and prints the resulting concatenated tuple.

Algorithm:

1. Start the program.
2. Create the first tuple, student1, containing the name, age, address, and college for the first student.
3. Create the second tuple, student2, containing the same information for the second student.
4. Print the contents of student1.
5. Print the contents of student2.
6. Concatenate student1 and student2 using the + operator and store the result in a new tuple called concatenated_tuples.
7. Print the final concatenated_tuples.
8. Stop the program.

Program:

```
# Creating the first tuple for student 1
```

```
student1 = ("Aarav Sharma", 20, "123 Maple St, Delhi", "ABC College of Engineering")
```

```
# Creating the second tuple for student 2
```

```
student2 = ("Diya Patel", 19, "456 Oak Ave, Mumbai", "XYZ Institute of Technology")
```

```
# Printing the individual tuples
```

```
print("--- Original Tuples ---")
```

```
print("Student 1 Data:", student1)
```

```
print("Student 2 Data:", student2)
```

```
print("-" * 25)
```

```
# Concatenating the two tuples

concatenated_tuples = student1 + student2


# Printing the concatenated tuple

print("\n--- Concatenated Tuple ---")

print("Combined Data:", concatenated_tuples)

print("-" * 25)
```

Output:

Result:

Experiment 14: Vowel Count in a String

Aim: To write a Python program that counts the total number of vowels (both uppercase and lowercase) present in a given string without using any control flow statements like if, for, or while.

Algorithm:

1. Start the program.
2. Initialize a string variable, input_string, with a sentence.
3. Create a string vowels containing all vowels ('aeiouAEIOU').
4. Use a generator expression (c in vowels for c in input_string) to produce a sequence of boolean values: True if a character is a vowel, and False otherwise.
5. Use the sum() function on this generator expression. sum() will treat True as 1 and False as 0, effectively counting the vowels.
6. Store the result in a variable named vowel_count.
7. Print the original string and the calculated vowel count.
8. Stop the program.

Program:

```
# Input string

input_string = "Welcome to Python Programming Lab"
```

```
# String containing all vowels
vowels = "aeiouAEIOU"

# Count vowels using sum() with a generator expression
# This avoids using explicit loops or conditional statements.
# The expression (1 for char in input_string if char in vowels) would also work.
vowel_count = sum(char in vowels for char in input_string)

# Printing the original string and the vowel count
print("--- Vowel Counter ---")
print("The given string is:", input_string)
print("The total number of vowels is:", vowel_count)
print("-" * 25)
```

Output:

Result:

Experiment 15: Check for Key in a Dictionary

Aim: To write a Python program to create a dictionary and then check if a specific, user-provided key exists within it.

Algorithm:

1. Start the program.
2. Create a dictionary `student_grades` with some sample key-value pairs (e.g., student names as keys and their grades as values).
3. Print the existing dictionary to the user.
4. Prompt the user to enter a key (a name) to search for.
5. Use the `in` membership operator to check if the entered key exists in the dictionary's keys.
6. Print a message indicating whether the key was found or not based on the result of the check.
7. Stop the program.

Program:

Creating a sample dictionary

```
student_grades = {
```

```
    "Rohan": 85,
```

```
    "Priya": 92,
```

```
    "Samuel": 78,
```

```
    "Fatima": 88
```

```
}
```

```
print("--- Key Existence Check in Dictionary ---")
```

```
print("Existing Dictionary:", student_grades)
```

```
print("-" * 35)
```

Key to be checked

```
key_to_check = input("Enter the name (key) to check for: ")
```

Check if the key exists using the 'in' operator

```
if key_to_check in student_grades:
```

```
    print(f"\nResult: Yes, the key '{key_to_check}' exists in the dictionary.")
```

```
    print(f"The value for '{key_to_check}' is {student_grades[key_to_check]}.")
```

```
else:
```

```
    print(f"\nResult: No, the key '{key_to_check}' does not exist in the dictionary.")
```

```
print("-" * 35)
```

Output:

Result:

Experiment 16: Add a New Key-Value Pair to a Dictionary

Aim: To write a Python program that adds a new key-value pair to an already existing dictionary and displays the dictionary before and after the addition.

Algorithm:

1. Start the program.
2. Initialize a dictionary `product_prices` with some products and their prices.
3. Print the original dictionary.
4. Define a new key (e.g., 'Keyboard') and a corresponding value (e.g., 800) to be added.
5. Add the new key-value pair to the dictionary using the syntax `dictionary_name[new_key] = new_value`.
6. Print the dictionary again to show that it has been updated.
7. Stop the program.

Program:

code Python

downloadcontent_copy

expand_less

IGNORE_WHEN_COPYING_START

IGNORE_WHEN_COPYING_END

```
# 16. Write a program to add a new key-value pair to an existing dictionary.
```

```
# Creating an existing dictionary of product prices
```

```
product_prices = {
```

```
    "Mouse": 450,
```

```
    "Monitor": 8000,
```

```
    "Webcam": 2500
```

```
}
```

```
print("--- Adding an Item to a Dictionary ---")
```

```
print("Original Dictionary:", product_prices)
```

```
print("-" * 35)
```

```
# New key and value to be added

new_product = "Keyboard"

new_price = 800


# Adding the new key-value pair

product_prices[new_product] = new_price


# Printing the updated dictionary

print("Updated Dictionary:", product_prices)

print(f"\nSuccessfully added '{new_product}' with price {new_price}.")

print("-" * 35)
```

Output:

Result:

Experiment 17: Sum of Items in a Dictionary

Aim: To write a Python program that calculates and prints the sum of all the values in a given dictionary.

Algorithm:

1. Start the program.
2. Create a dictionary `monthly_expenses` where keys are expense categories and values are the amounts.
3. Print the dictionary.
4. Use the `.values()` method to get a view object containing all the values from the dictionary.
5. Use the built-in `sum()` function to calculate the sum of all elements in the values view object.
6. Store the result in a variable `total_sum`.
7. Print the calculated total sum.
8. Stop the program.

Program:

A dictionary with items and their costs

```
monthly_expenses = {  
    "Rent": 15000,  
    "Groceries": 4500,  
    "Transport": 1200,  
    "Utilities": 2300  
}
```

```
print("--- Summing Dictionary Values ---")
```

```
print("Dictionary of Expenses:", monthly_expenses)
```

```
print("-" * 35)
```

Get all the values from the dictionary using .values()

```
all_values = monthly_expenses.values()
```

Calculate the sum of these values

```
total_sum = sum(all_values)
```

Print the final sum

```
print("The sum of all items (values) in the dictionary is:", total_sum)
```

```
print("-" * 35)
```

Output:

Result:
